

bit_analysis_1m

July 10, 2025

1 Análisis de eficiencia de bits 1 metro de distancia

Este notebook analiza qué tan parecidos son los bits recibidos `bits_rx` con los bits transmitidos `bits_tx` y presenta gráficas, métricas de rendimiento y las imágenes que no cumplen con el criterio de coincidencia.

```
[1]: import pandas as pd
from pathlib import Path
from IPython.display import display, Image
import matplotlib.pyplot as plt

path = "./results/1m/"
nombre_csv = "resultados_comparacion_1m.csv"
df = pd.read_csv(path + nombre_csv)

# Vista rápida del DataFrame
display(df.head())
print(f"Filas totales: {len(df)}")
```

	num_auto	bw_plate_tx \
0	1	dataset_yolo/autos_bw/auto001.png
1	1	dataset_yolo/autos_bw/auto001.png
2	1	dataset_yolo/autos_bw/auto001.png
3	1	dataset_yolo/autos_bw/auto001.png
4	1	dataset_yolo/autos_bw/auto001.png

	bw_plate_rx	ocr_txt_tx	bits_equal	bit_errors \
0	data_1m/autos_bw/auto001_rep01.png	K0Z.808	False	2
1	data_1m/autos_bw/auto001_rep02.png	K0Z.808	True	0
2	data_1m/autos_bw/auto001_rep03.png	K0Z.808	True	0
3	data_1m/autos_bw/auto001_rep04.png	K0Z.808	True	0
4	data_1m/autos_bw/auto001_rep05.png	K0Z.808	True	0

	bits_tx	bits_rx
0	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
1	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
2	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
3	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]

```
4  [0 0 1 ... 0 0 0]  [0 0 1 ... 0 0 0]
```

Filas totales: 1520

1.1 Distribución de errores de bit por imagen

```
[2]: plt.figure()  
df['bit_errors'].plot(kind='hist', bins=30)  
plt.xlabel('Errores de bit por imagen')  
plt.ylabel('Frecuencia')  
plt.title('Histograma de errores de bit')  
plt.show()
```



1.2 Errores de bit por réplica

```
[3]: plt.figure(figsize=(10,4))  
plt.plot(df.index, df['bit_errors'], marker='o', linestyle='none')  
plt.xlabel('Índice de fila')  
plt.ylabel('Errores de bit')  
plt.title('Errores de bit por réplica')
```

```
plt.show()
```



1.3 Imágenes que no cumplen el criterio

```
[4]: # Eliminamos las que tienen 2000 errores, ya que es la imagen inversa
df = df[df['bit_errors'] < 2000]
```

```
[5]: import numpy as np    # por si no estaba importado

# --- Parámetros de tolerancia ---
BITS_TOTAL = 2000
max_errors = 100 # máximo de bits erróneos permitidos
max_ber     = max_errors / BITS_TOTAL # % BER máximo

# --- BER por fila ---
df['bits_total'] = BITS_TOTAL
df['ber'] = df['bit_errors'] / BITS_TOTAL

# --- Criterio de falla ---
fail_df = df[(df['bit_errors'] > max_errors) | (df['ber'] > max_ber)]

print(f'Imágenes que superan el umbral: {len(fail_df)}')
print(f'Total de imágenes: {len(df)}')
print(f'Porcentaje de fallos: {len(fail_df) / len(df) * 100:.2f}%')
display(
    fail_df[['num_auto', 'bw_plate_tx', 'bw_plate_rx',
             'bit_errors', 'ber']]
)
```

Imágenes que superan el umbral: 66

Total de imágenes: 1514
Porcentaje de fallos: 4.36%

	num_auto	bw_plate_tx \		
42	13	dataset_yolo/autos_bw/auto013.png		
58	128	dataset_yolo/autos_bw/auto128.png		
59	128	dataset_yolo/autos_bw/auto128.png		
89	26	dataset_yolo/autos_bw/auto026.png		
134	64	dataset_yolo/autos_bw/auto064.png		
...	...	...		
1505	26	dataset_yolo/autos_bw/auto026.png		
1506	26	dataset_yolo/autos_bw/auto026.png		
1515	26	dataset_yolo/autos_bw/auto026.png		
1517	26	dataset_yolo/autos_bw/auto026.png		
1518	26	dataset_yolo/autos_bw/auto026.png		
		bw_plate_rx	bit_errors	ber
42		data_1m/autos_bw/auto013_rep01.png	789	0.3945
58		data_1m/autos_bw/auto128_rep01.png	902	0.4510
59		data_1m/autos_bw/auto128_rep02.png	902	0.4510
89		data_1m/autos_bw/auto026_rep01.png	863	0.4315
134		data_1m/autos_bw/auto064_rep01.png	793	0.3965
...	...	...	...	...
1505		data_1m/autos_bw/auto026_rep17.png	773	0.3865
1506		data_1m/autos_bw/auto026_rep18.png	773	0.3865
1515		data_1m/autos_bw/auto026_rep19.png	773	0.3865
1517		data_1m/autos_bw/auto026_rep20.png	773	0.3865
1518		data_1m/autos_bw/auto026_rep21.png	773	0.3865

```
[66 rows x 5 columns]
```

1.3.1 Visualización de las primeras 5 imágenes fallidas

```
[6]: path = 'results/1m/'

for _, row in fail_df.head(25).iterrows():
    print(f"Auto {row['num_auto']} - errores: {row['bit_errors']}")
    display(Image(filename=row['bw_plate_tx']))
    display(Image(filename=path + row['bw_plate_rx']))
```

Auto 13 - errores: 789





Auto 128 - errores: 902



Auto 128 - errores: 902



Auto 26 - errores: 863



Auto 64 - errores: 793



Auto 61 - errores: 960



Auto 53 - errores: 945



Auto 104 - errores: 952





Auto 105 - errores: 808



Auto 107 - errores: 820



Auto 107 - errores: 823



Auto 107 - errores: 823



Auto 61 - errores: 623



Auto 89 - errores: 1032



Auto 89 - errores: 1033



Auto 73 - errores: 982



Auto 77 - errores: 558



Auto 36 - errores: 814





Auto 36 - errores: 812



Auto 65 - errores: 753



Auto 65 - errores: 752



Auto 65 - errores: 749



Auto 80 - errores: 724



Auto 80 - errores: 724



Auto 64 - errores: 734





```
[7]: # --- Umbral de bits erróneos ---
thr_err = 15

# --- Agrupar por auto y comprobar si alguna réplica cumple ---
auto_status = (
    df.groupby('num_auto')['bit_errors']
      .apply(lambda s: (s < thr_err).any())    # True = tiene al menos una buena
)

# Autos sin réplica "buena"
failed_autos = auto_status[~auto_status]      # False significa que falló
num_failed   = failed_autos.size
num_total    = auto_status.size

print(f'Autos sin ninguna réplica con < {thr_err} errores de bit: '
      f'{num_failed} de {num_total}')

# Mostrar la lista (si quieres verla)
display(failed_autos.index.tolist())
```

Autos sin ninguna réplica con < 15 errores de bit: 0 de 178

[]